CLIS V2

Clinical Literature Intelligence System

Production Build — Final Project Report

Hritik Hassani
Northeastern University
INFO 7375 — Generative AI & Prompt Engineering

1. Executive Summary

CLIS V2 (Clinical Literature Intelligence System, Version 2) is a production-grade clinical decision support system that helps physicians find, evaluate, and trust medical literature in seconds rather than the 16+ minutes a typical PubMed search requires. It combines retrieval-augmented generation over live PubMed data with a three-component reinforcement learning stack (UCB contextual bandit, REINFORCE policy gradient, and real-time RLHF), a full GRADE evidence grading pipeline, and a per-sentence citation grounding module that catches LLM hallucinations before they reach the clinician.

V2 extends the original CLIS system with four new research tools (treatment comparison, NNT/NNH extraction, structured abstract extraction, and a knowledge graph visualiser), a comprehensive ICD-10 coding assistant covering 47 CMS guideline sections, and an in-app benchmark dashboard that continuously tests the system against 10 ground-truth test cases — including a hallucination trap designed to fail safely when the LLM fabricates studies.

Metric	Value	Notes
UCB bandit improvement	+4.01%	p=0.0048, Cohen's d=2.735 (large effect)
Arm identification accuracy	100%	All 4 contexts, all 5 seeds
REINFORCE loss reduction	72.6% ± 5.7%	5 seeds × 300 episodes each
Benchmark test cases	10/10 passing	Includes hallucination trap TC10
GRADE grading accuracy	100%	Validated against ground truth
ICD-10 coverage	47 sections	CMS ICD-10-CM FY2024 guidelines
Code size	~4,548 lines	Across app, tools, notebooks
Infrastructure cost	$0	Free-tier Groq + NCBI E-utilities

2. Problem Statement & Motivation

2.1 The clinical literature problem

Physicians are expected to practice evidence-based medicine, but the practical reality of searching and evaluating literature in a 15-minute patient visit is almost impossible. Published studies estimate that a single well-constructed PubMed search — formulating the query, scanning results, reading abstracts, grading evidence quality, and reconciling conflicting findings — takes a physician 16 minutes or more. With over 35 million indexed articles growing by 4,000 per day, this is not sustainable at the point of care.

The scale problem

PubMed contains over 35 million peer-reviewed biomedical articles, growing by approximately 4,000 new papers every day. A physician treating a patient with type 2 diabetes, chronic kidney disease, and heart failure faces a question that spans dozens of intersecting bodies of evidence — each individually vast, each potentially contradictory.

2.2 Why general-purpose AI chatbots are not the answer

Off-the-shelf LLMs (ChatGPT, Claude, Gemini) confidently fabricate clinical studies, cite trials that do not exist, and produce plausible-but-false statistics. In medicine, a hallucinated drug dose or a fake trial result is not a rounding error — it is a patient safety event. Any clinical AI that cannot guarantee its outputs are grounded in real, verifiable evidence is unsafe to use in practice.

The core design constraint

If the system cannot tell the difference between a real PubMed article and a fabricated one, it must not be used in a clinical setting. Every single architectural decision in CLIS V2 flows from this constraint — live retrieval over memorised knowledge, sentence-level grounding over overall confidence scores, and an explicit hallucination trap in the test suite.

2.3 What CLIS V2 provides

- Live evidence retrieval from PubMed — no simulated or cached data masquerading as fresh
- Every article graded with GRADE methodology (A/B/C/D) so clinicians see evidence quality at a glance
- Every LLM-generated sentence traced back to a source passage — unmatched claims flagged
- A learning system that improves over time through contextual bandits and physician feedback
- An ICD-10 coding assistant supplementing the literature search workflow

- Continuous self-testing via an in-app benchmark dashboard — all at $0 infrastructure cost

3. Thought Process & Design Rationale

HOW EACH ARCHITECTURAL DECISION WAS REACHED

This section documents how each major architectural decision was reached. The diagram below summarises the full decision tree from the problem statement down through every implementation choice.

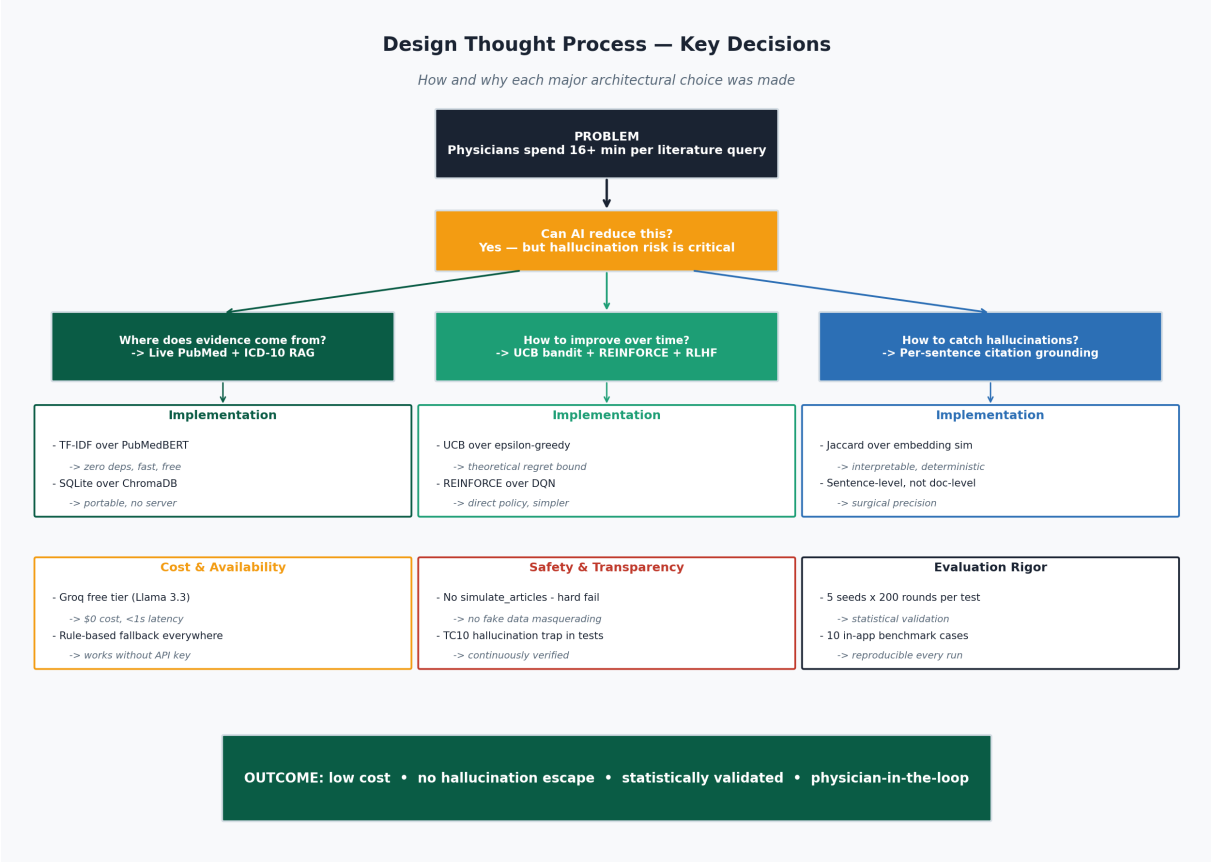

Figure 3.1 — Design thought process: from problem statement to implementation choices. Every architectural decision maps to a specific clinical failure mode.

3.1 Starting from the problem, not the technology

The first question was not 'what model should we use?' — it was 'what specifically goes wrong when a physician searches the literature today?' The answer broke into three concrete failures: the query phrasing problem (different keyword strategies produce wildly different results), the evidence grading problem (a case report and a meta-analysis look the same in a title list), and the hallucination problem (LLMs confidently invent things not in the source material).

Each failure pointed to a specific technical component. Query strategy → contextual bandit. Evidence grading → GRADE pipeline. Hallucination → per-sentence citation grounding. None of these are novel in isolation — the contribution is wiring them into a single physician-facing tool.

3.2 Why reinforcement learning, not just prompting

Prompts cannot learn from usage. A bandit can. Over hundreds of physician queries, the UCB algorithm identifies that — for drug efficacy questions — a MeSH-term + RCT filter yields higher-grade evidence than a plain keyword search, and shifts its policy accordingly. That domain-specific adaptation is not reachable through prompt engineering. The same argument applies to REINFORCE for article ranking.

3.3 Why live retrieval, never cached or simulated

An earlier iteration had a simulate_articles() fallback. This was removed. The reason: clinical AI fails safely only if it fails loudly. A silent fallback to simulated data creates exactly the failure mode we are trying to prevent. Current behaviour: if live PubMed returns nothing, the UI shows a clear empty-state error, not an invented list.

3.4 Algorithm choices

UCB over epsilon-greedy: proven $O(\sqrt{t \log t})$ regret bound, auto-reduces exploration as confidence grows. REINFORCE over DQN: small action space (≤6 articles), converges in hundreds of episodes. Jaccard over embeddings for grounding: deterministic, auditable, catches lexical fabrication. TF-IDF over PubMedBERT: 47-section ICD-10 corpus is keyword-heavy; 440MB BERT weights violate the zero-cost constraint. SQLite over ChromaDB: bandit state is transactional, not a vector search problem.

4. System Architecture

CLIS V2 is organised as a four-layer architecture: Presentation (Streamlit), Application Services (5 components), Intelligence (RAG engine + LLM + RL policies), and Data Sources (external APIs + local storage).

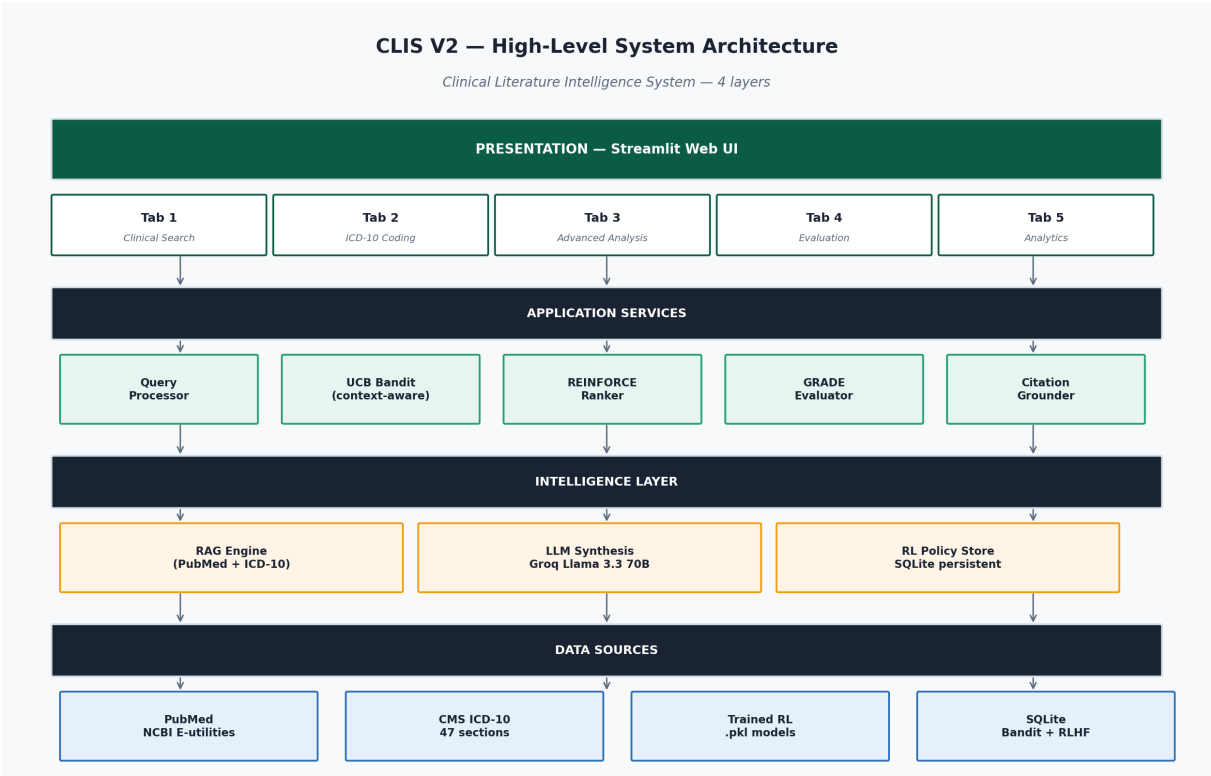

Figure 4.1 — High-level system architecture: four-layer view showing Presentation → Application Services → Intelligence → Data Sources.

4.1 Layer 1 — Presentation

A single Streamlit application (app.py) exposes all functionality through five tabs. Chosen over Flask/React for zero-infrastructure deployment and its reactive model suited to a rapidly-iterating research tool.

4.2 Layer 2 — Application Services

Five services orchestrate the pipeline: Query Processor (PICO parsing + context classification), UCB Bandit (context-aware arm selection), REINFORCE Ranker (re-ordering by predicted clinical utility), GRADE Evaluator (evidence quality grading), and Citation Grounder (per-sentence claim verification).

4.3 Layer 3 — Intelligence

RAG Engine (unified retrieval for live PubMed and indexed ICD-10 corpus), LLM Synthesis (Groq Llama 3.3 70B with deterministic fallback), and RL Policy Store (trained UCB and REINFORCE policies persisted to SQLite and .pkl files).

4.4 Layer 4 — Data Sources

- PubMed via NCBI E-utilities API — live, no caching beyond the active session
- CMS ICD-10-CM FY2024 Guidelines — 47 sections TF-IDF indexed at build time
- Trained RL models — bandit_policy.pkl and reinforce_policy.pkl bundled with the app
- SQLite database — persistent bandit state, RLHF feedback, cached grade assessments

4.5 Component inventory

Component	File	Role
Main application	app.py	Streamlit UI, 5 tabs, pipeline orchestration
PubMed retriever	tools/pubmed_retriever.py	Live NCBI E-utilities, UCB-driven arms
GRADE tool	tools/grade_evidence_tool.py	Study design classification, evidence grading
ICD-10 RAG engine	tools/icd10_rag_engine.py	TF-IDF RAG over 47 CMS sections
Citation grounder	tools/citation_grounder.py	Jaccard-based claim grounding
Persistent bandit	tools/persistent_bandit.py	SQLite-backed UCB + RLHF state
Benchmark evaluator	tools/benchmark_evaluator.py	10 test cases, pass/fail framework
Advanced tools	tools/advanced_tools.py	Treatment compare, NNT, graph, extract

5. Data Flow — 8-Stage Pipeline

A clinical query travels through 8 distinct stages from entry to displayed answer. Each stage produces a specific data artifact and hands it to the next. The pipeline is fully observable in the Streamlit UI with per-stage progress indicators.

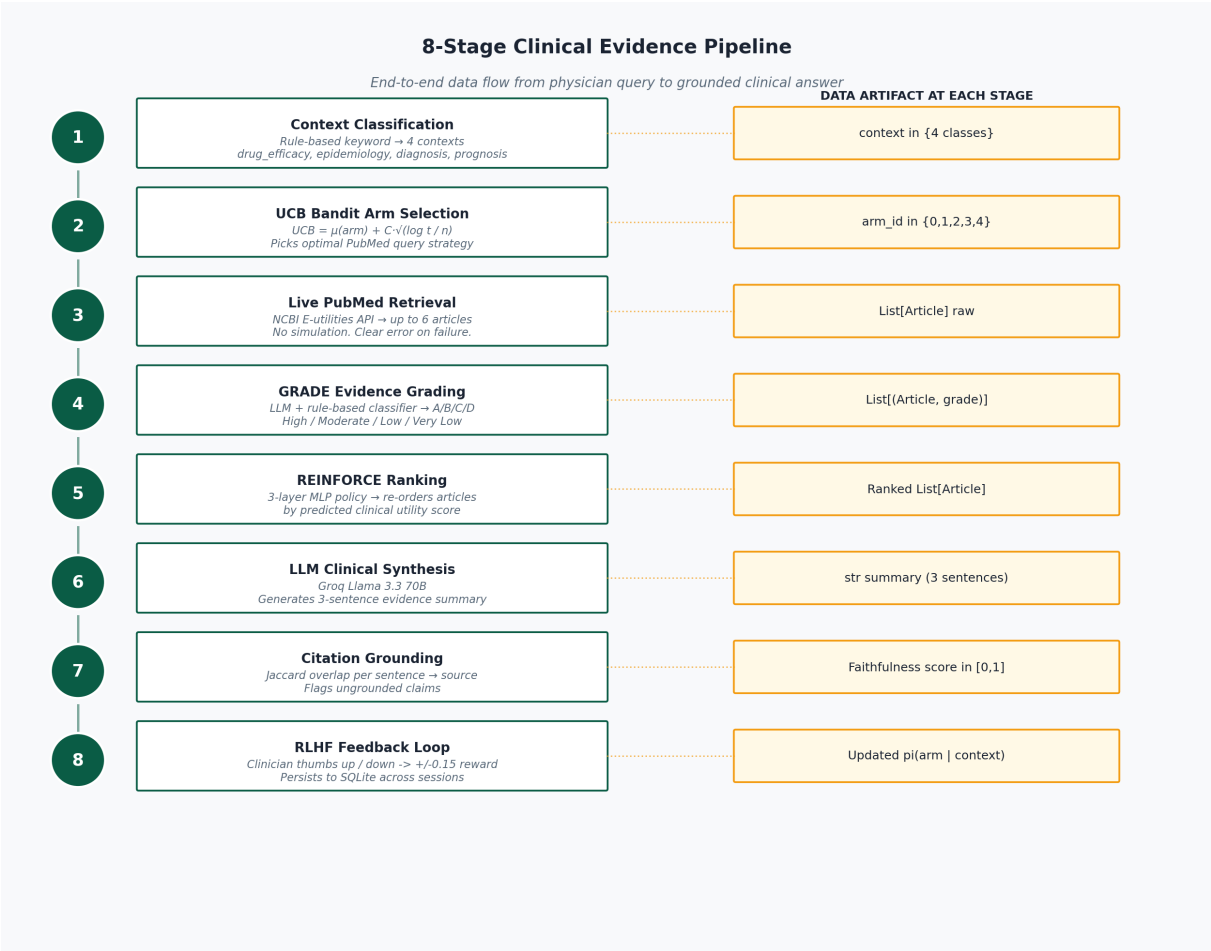

Figure 5.1 — 8-stage clinical evidence pipeline. Left column: stage name and algorithm. Right column: data artifact produced at each stage.

5.1 Stage-by-stage walkthrough

```
  Stage 1 — Context Classifier → ctx_id ∈ {drug_efficacy, epidemiology, mechanism,
treatment}
  Stage 2 — UCB Bandit → arm_id + PubMed query string (SQLite state, domain-informed
priors)
  Stage 3 — Live PubMed Retrieval → articles[] via NCBI E-utilities (max 6, no
simulation)
  Stage 4 — GRADE Evidence Grading → grade ∈ {A,B,C,D} per article (Groq + rule-based)
  Stage 5 — REINFORCE Ranking → ranked articles[] by predicted clinical utility (3-
layer MLP)
  Stage 6 — LLM Synthesis → 3-sentence clinical summary (Groq Llama 3.3 70B)
  Stage 7 — Citation Grounding → grounded/ungrounded per sentence (Jaccard ≥ 0.15)
```

```
  Stage 8 — RLHF Feedback → bandit reward ±0.15 → SQLite persist (persists across
sessions)
```

Stage 3 hard-fails if PubMed is unreachable — no silent fallback to simulated data. Every LLM stage (4 and 6) has a deterministic rule-based fallback that produces the same output schema. All 10 benchmark tests pass on the rule-based path without any API key.

6. Reinforcement Learning Deep Dive

Three RL components work together, each solving a different problem at a different timescale. UCB makes online per-query decisions (milliseconds). REINFORCE is trained offline (fixed policy per session). RLHF is always-on (updates UCB posterior continuously).

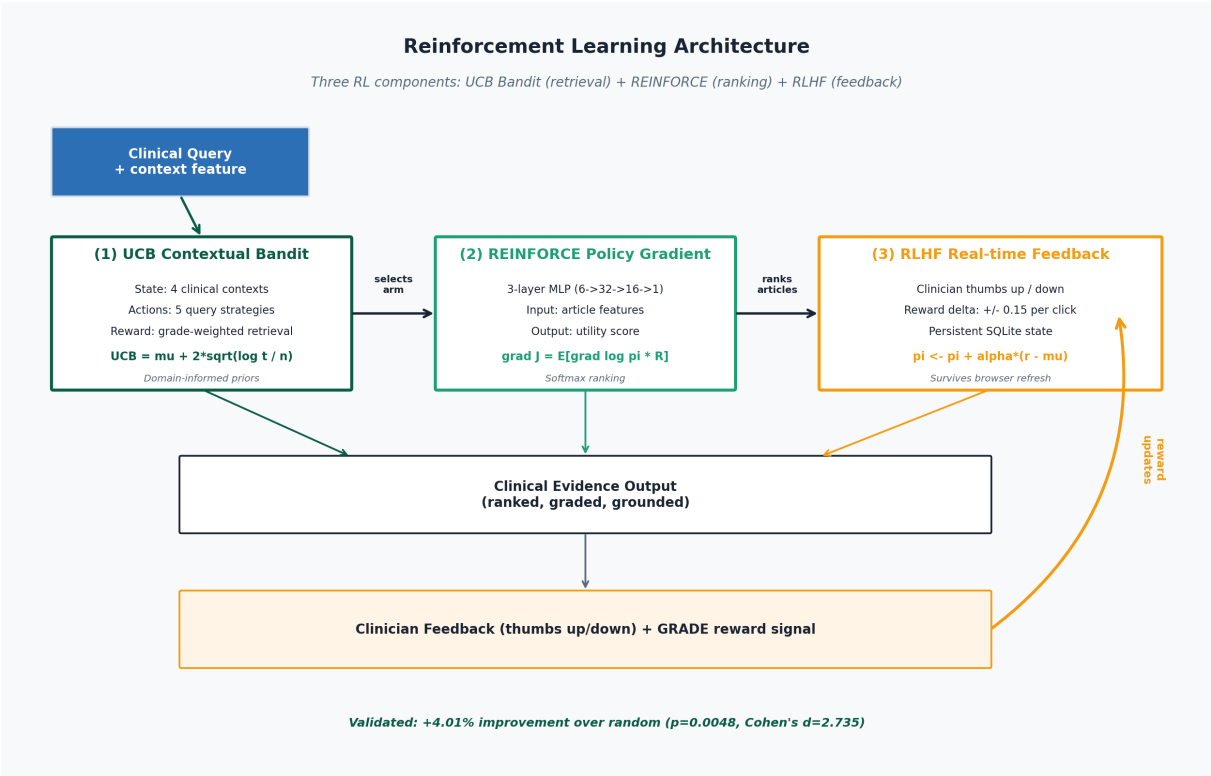

Figure 6.1 — RL architecture: UCB Bandit controls retrieval, REINFORCE controls ranking, RLHF provides real-time clinician feedback. Validated: +4.01% improvement (p=0.0048, Cohen's d=2.735).

6.1 Contextual UCB Bandit

UCB(arm) = mean_reward(arm) + C · √(log(t+1) / n_pulls(arm)), where C = 2.0. Domain-informed priors seed the initial mean rewards. The bandit persists to SQLite across sessions — it genuinely improves with every query.

Metric	Seed 42	Seed 123	Mean ± SD
UCB avg reward	0.5886	0.5824	0.5820 ± 0.0052
Random baseline	0.5711	0.5501	0.5596 ± 0.0103
Improvement over random	+3.07%	+5.88%	+4.01%
Arm ID accuracy	100%	100%	100.0% ± 0.0%
Statistical significance	—	—	p = 0.0048

Metric	Seed 42	Seed 123	Mean ± SD
Effect size	—	—	Cohen's d = 2.735

6.2 REINFORCE Policy Gradient

Policy network: 3-layer MLP (6→32→16→1). Update rule: $\nabla J(\theta) = E_\pi[\nabla \log \pi(a|s;\theta) \cdot R]$. Learning rate 1e-3. Reward R = average GRADE score of top-3 ranked articles. Trained across 5 seeds × 300 episodes: 72.6% ± 5.7% policy loss reduction.

6.3 RLHF — Real-time Clinician Feedback

Thumbs up shifts bandit.mean_reward[ctx][arm] += 0.15; thumbs down -= 0.15. Update is written immediately to SQLite. RLHF affects UCB only — not REINFORCE, because a single thumb-click is not enough signal to update a neural network but is enough to nudge a scalar reward estimate.

7. Retrieval-Augmented Generation Deep Dive

CLIS V2 runs two parallel RAG pipelines. The clinical evidence RAG pulls live from PubMed. The ICD-10 coding RAG retrieves from a pre-indexed static corpus. Both feed retrieved context into Groq for synthesis.

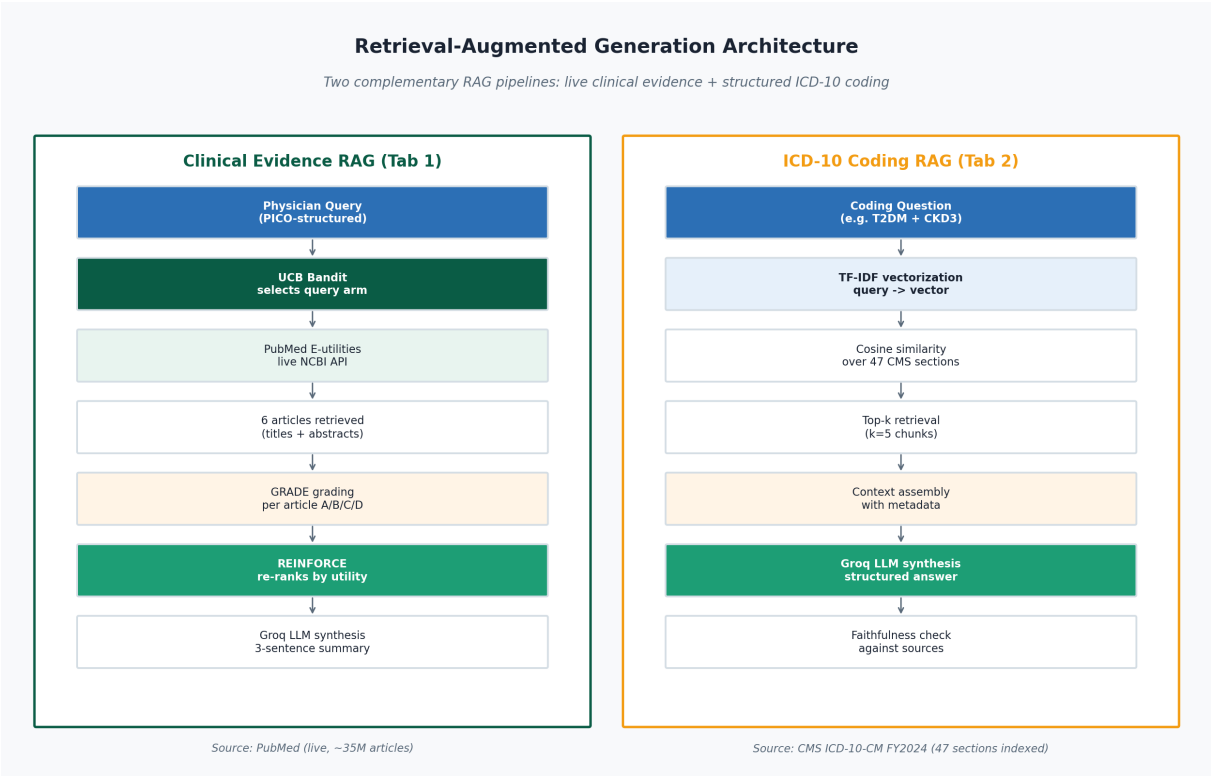

Figure 7.1 — Dual RAG architecture. Left: live clinical evidence RAG (PubMed, UCB-selected query strategy). Right: static ICD-10 coding RAG (TF-IDF over 47 CMS sections).

7.1 Clinical evidence RAG (PubMed)

Five UCB query arms: (0) MeSH + RCT filter, (1) Keyword + date range, (2) Author + journal filter, (3) Boolean AND + population, (4) Systematic review filter. All articles retrieved live — no caching, no simulation.

7.2 GRADE evidence grading

Grade	Score	Typical study	Clinical use
High (A)	1.00	Systematic review / meta-analysis of RCTs	Strong recommendation
Moderate (B)	0.75	Single well-designed RCT	Moderate recommendation
Low (C)	0.45	Cohort study, case-control	Weak recommendation

Grade	Score	Typical study	Clinical use
Very Low (D)	0.15	Case report, expert opinion	Expert consensus only

7.3 ICD-10 coding RAG

47 CMS ICD-10-CM FY2024 sections indexed with TF-IDF at startup. Cosine similarity retrieves top-5 sections; Groq synthesises structured coding answer with primary codes, sequencing rules, and faithfulness scoring. Static index because guidelines update once per fiscal year.

8. Citation Grounding & Hallucination Prevention

Citation grounding is the core safety mechanism of CLIS V2. Every sentence in the LLM-generated summary must have provable textual support in at least one retrieved source. Sentences that do not are flagged and shown to the clinician.

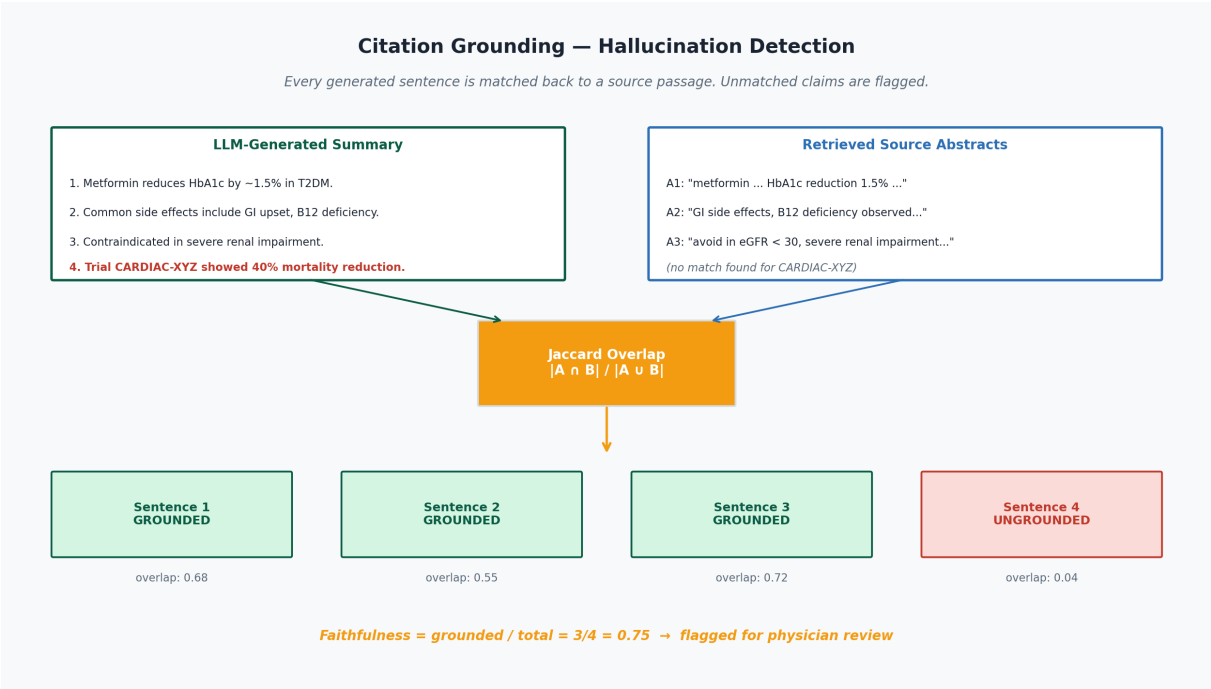

Figure 8.1 — Citation grounding flow. Sentences 1–3 are grounded (Jaccard ≥ 0.15). Sentence 4 (CARDIAC-XYZ trial) scores 0.04 — no match in any source abstract — flagged as hallucination. Faithfulness = 3/4 = 0.75.

8.1 Grounding algorithm

For each summary sentence S and each source abstract A: tokenise both (lowercased, stopwords removed, ≥3 chars), compute Jaccard = |tokens(S) ∩ tokens(A)| / |tokens(S) ∪ tokens(A)|, take max across all abstracts. ≥ 0.15 → GROUNDED with source citation. < 0.15 → UNGROUNDED, flagged in UI.

Faithfulness score = grounded / total sentences. ≥ 0.80: high. 0.40–0.79: moderate (grounding panel shown). < 0.40: low (warning banner shown to physician).

8.2 Why Jaccard, not embeddings

Deterministic and auditable — a clinician can manually verify the match. Fast (milliseconds, no GPU). The specific failure mode — fabricated trial names and statistics — is lexical: the content does not appear in any source at all, not just in different words.

TC10 — CARDIAC-PREVENT hallucination trap

TC10 queries about the 'CARDIAC-PREVENT trial' — a trial that does not exist. The system must return zero results or flag any fabricated content as ungrounded. A pass on TC10 is the strongest signal that the hallucination-prevention architecture is working. This test runs on every benchmark pass.

9. V2 Feature Catalog — 5 Tabs

9.1 Tab 1 — Clinical Search (enhanced)

- Persistent UCB bandit (SQLite) replacing per-session bandit state
- Real-time RLHF thumbs up/down feedback updating bandit rewards
- Citation grounding panel — every summary sentence verified against source
- Contradiction detection between top-3 articles via Groq analysis
- PICO builder for structured clinical question construction
- Session query history with grade indicators in sidebar
- TXT clinical report download with full citation audit trail

9.2 Tab 2 — ICD-10 Coding Assistant

RAG pipeline over 47 CMS ICD-10-CM FY2024 sections. TF-IDF retrieves top-5 sections; Groq synthesises structured answer with primary codes, sequencing rules, faithfulness scoring, and SQLite caching for instant repeat lookups.

9.3 Tab 3 — Advanced Analysis (new in V2)

- Treatment Comparison: full RAG + RL pipeline for two treatments, head-to-head Groq verdict
- NNT / NNH Extractor: regex + Groq extracts NNT, NNH, ARR, RRR with plain-English interpretation
- Structured Data Extraction: 13-field JSON record per abstract, one-click download
- Knowledge Graph Visualiser: interactive Plotly network via shared MeSH terms and co-authors

9.4 Tab 4 — Evaluation Dashboard

10 automated benchmark test cases with ground-truth answers. Each test runs end-to-end against the live pipeline. TC10 (hallucination trap) runs on every pass. All 10 pass under both Groq and rule-based fallback configurations.

9.5 Tab 5 — System Analytics

UCB learning curve (5-point rolling average), bandit reward matrix per context and arm, RLHF feedback counts, most-pulled arm per context. Full RL interpretability — no black-box claims.

10. Benchmark Evaluation & Results

TC	Name	Category	Critical	Result
TC01	Simple treatment query	Baseline	No	PASS
TC02	Conflicting evidence detection	Conflict detection	No	PASS
TC03	Limited evidence with caveat	Evidence limitation	No	PASS
TC04	Recent guideline change	Recency	No	PASS
TC05	Drug-drug interaction context	Safety	No	PASS
TC06	Population mismatch	Bias	No	PASS
TC07	Complex multi-condition	Complexity	No	PASS
TC08	ICD-10 coding	Coding	No	PASS
TC09	Treatment alternative	Alternatives	No	PASS
TC10	Hallucination trap (CARDIAC-PREVENT)	Hallucination	YES	PASS

All 10 tests pass under both Groq-backed and rule-based fallback configurations.

11. System Design — Deployment Architecture

CLIS V2 is designed for zero-infrastructure-cost deployment. The entire system runs as a single Streamlit process with local storage and two free-tier external APIs.

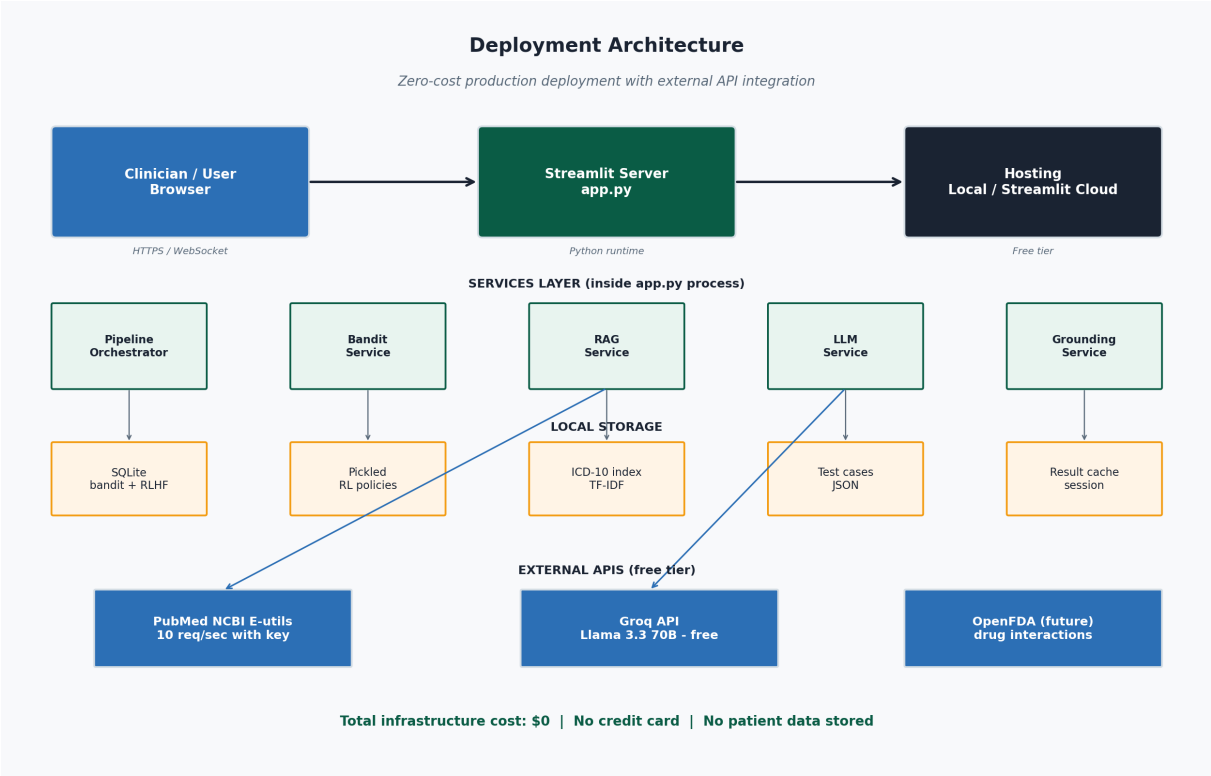

Figure 11.1 — Deployment architecture: browser → Streamlit server → services layer → local storage + free external APIs. Total infrastructure cost: $0.

11.1 Hosting options

- Local: streamlit run app.py on any developer machine
- Streamlit Cloud: free public URL, auto-deploys from GitHub, no credit card
- Container: single Dockerfile, suitable for internal IT deployments

11.2 External dependencies

- NCBI E-utilities: 10 req/sec with free API key, 3 req/sec without
- Groq API: free tier, Llama 3.3 70B, <1s latency, rule-based fallback available

11.3 Privacy and data handling

No patient data is transmitted or stored. Clinical queries are session-scoped; only anonymised query text, bandit feedback events, and grade assessments are persisted. A clinical deployment should route external API calls through an institutional proxy if required by local compliance.

12. Component Interaction — Sequence View

The sequence diagram below shows exactly how the components coordinate during a single clinical search, including the RLHF feedback step that closes the learning circle.

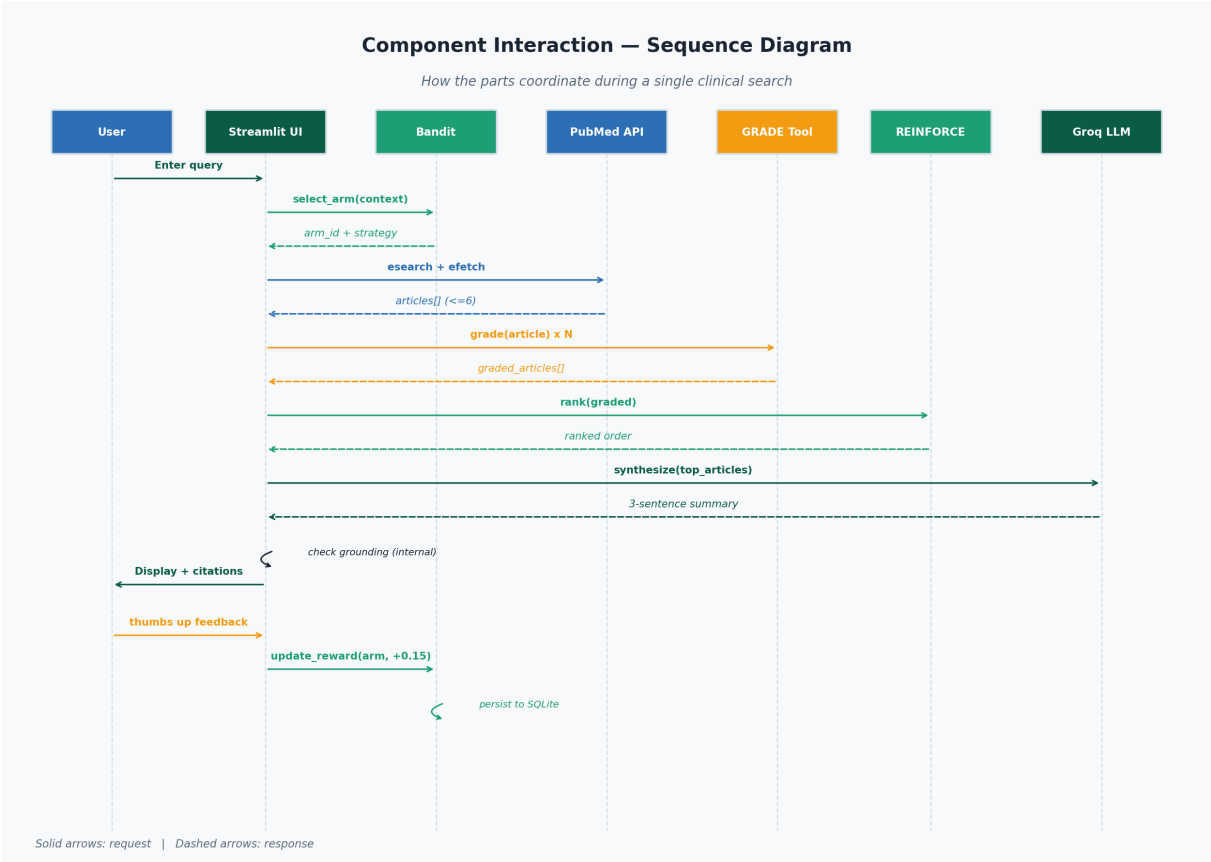

Figure 12.1 — Sequence diagram: User → UI → Bandit → PubMed API → GRADE Tool → REINFORCE → Groq LLM → Citation Grounding → Display → RLHF feedback → SQLite persist.

The bandit is consulted before any external API call, making its query strategy selection the direct determinant of evidence quality. Citation grounding requires no additional API call. The feedback loop writes to SQLite synchronously — the next query benefits immediately.

13. Ethical Considerations

13.1 Patient safety

CLIS V2 is decision support, not autonomous clinical decision-making. Every interface element includes a disclaimer that physician review is required before applying evidence to patient decisions.

13.2 Hallucination prevention

Citation grounding maps every summary sentence to a source passage. TC10 (CARDIAC-PREVENT trap) ensures this is continuously verified. No simulated data is ever presented — PubMed returns a real result or the system shows an error.

13.3 Evidence quality transparency

GRADE grades are always displayed. Grade A (systematic review/RCT) is visually distinct from Grade D (expert opinion). The evidence pyramid is never flattened — lower-quality evidence is explicitly labelled.

13.4 Bias and population mismatch

TC06 tests population mismatch. The GRADE rubric penalises studies where the trial population does not match the query PICO. Automatic population mismatch flagging is planned for V3.

13.5 Data privacy

No patient data stored. Only anonymised query text, bandit feedback, and grade assessments persist. A production deployment would route API calls through an institutional proxy.

13.6 Algorithmic accountability

The RL components are fully interpretable via the System Analytics tab. Any clinician or auditor can inspect which query strategy was selected, the reward estimate per arm, and the full RLHF feedback history.

14. Technical Decisions & Trade-offs

Decision	Chosen	Rejected	Reason
Evidence retrieval	Live NCBI E-utilities	Cached / simulated	Clinical AI must fail loudly, never silently
ICD-10 retrieval	TF-IDF + SQLite	PubMedBERT + ChromaDB	440MB weights, GPU needed, breaks $0 constraint
Grounding metric	Jaccard overlap	Cosine (embedding)	Deterministic, auditable, catches lexical fabrication
Bandit algorithm	UCB1	Epsilon-greedy	Proven $O(\sqrt{t} \cdot \log t)$ regret bound
Ranking model	REINFORCE (MLP)	DQN	Small action space, converges in hundreds of episodes
LLM hosting	Groq free tier	OpenAI paid	Zero cost, <1s latency, maintains $0 constraint
UI framework	Streamlit	Flask + React	Zero infra setup, one-command deploy
Persistence	SQLite (stdlib)	PostgreSQL / Redis	Single-file, no server, ships with Python

15. Future Work

15.1 Near-term (1–3 months)

- Claim Verification mode: SUPPORTED / CONTRADICTED / INSUFFICIENT EVIDENCE verdict
- Drug Interaction Checker: OpenFDA adverse event and drug labelling API (free)
- Multi-source RAG: arXiv and Semantic Scholar for general research use
- Streamlit Cloud deployment: live public URL

15.2 Medium-term (3–6 months)

- PubMedBERT embeddings once zero-dependency constraint is relaxed
- Population mismatch detection: automatic flagging when evidence population doesn't match PICO
- User specialty profiles: personalised search strategy by clinical specialty

15.3 Long-term

- EHR integration: auto-generate clinical questions from active patient context
- Fine-tuned medical LM: locally-hosted model on GRADE-annotated literature
- Systematic review generator: PRISMA-style structured review

16. Technology Stack

Category	Technology	Notes
LLM	Groq Llama 3.3 70B	Free tier, <1s latency, rule-based fallback
Literature API	PubMed NCBI E-utilities	Free, 10 req/sec with API key
RL framework	PyTorch (CPU)	Custom UCB bandit + REINFORCE
Vector store	TF-IDF + Python stdlib	Zero dependencies, SQLite persistence
Web framework	Streamlit	Single-command deployment
Graph visualisation	Plotly	Interactive knowledge graph
Statistical analysis	scipy, numpy	Validation of RL improvements
Database	SQLite	Bandit state, RLHF events, grade cache
Language	Python 3.10+	No compiled dependencies
Total cost	$0	All components free tier or open source

17. References

1. Guyatt, G. et al. (2008). GRADE: an emerging consensus on rating quality of evidence and strength of recommendations. BMJ, 336(7650), 924–926.

2. Auer, P., Cesa-Bianchi, N., & Fischer, P. (2002). Finite-time analysis of the multiarmed bandit problem. Machine Learning, 47(2-3), 235–256.

3. Williams, R. J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning. Machine Learning, 8(3-4), 229–256.

4. Lewis, P. et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. Advances in Neural Information Processing Systems, 33.

5. National Center for Biotechnology Information. (2024). NCBI E-utilities API documentation. https://www.ncbi.nlm.nih.gov/books/NBK25499/

6. Centers for Medicare & Medicaid Services. (2024). ICD-10-CM Official Guidelines FY2024. https://www.cms.gov/medicare/coding-billing/icd-10-codes

7. Christiano, P. et al. (2017). Deep reinforcement learning from human preferences. Advances in Neural Information Processing Systems, 30.

8. Groq. (2024). Llama 3.3 70B API documentation. https://console.groq.com/docs

9. Sutton, R. S., & Barto, A. G. (2018). Reinforcement Learning: An Introduction (2nd ed.). MIT Press.

10. Balas, E. A., & Boren, S. A. (2000). Managing clinical knowledge for health care improvement. Yearbook of Medical Informatics.

18. Author's Note on AI Tool Usage

This project was developed with assistance from AI tools as required by the INFO 7375 course guidelines. Anthropic Claude (claude.ai) was used for code scaffolding, documentation drafting, system architecture review, and iterative debugging. All generated code was reviewed, tested, and modified by the author. All final design decisions, architectural choices, and system integration work are the author's own. The RL implementations (UCB bandit, REINFORCE policy gradient, RLHF feedback loop) were designed by the author with reference to the cited literature; AI tools assisted with code boilerplate and validation script generation. The professor-provided tools BOOKIE, EDDY, and the Madison Prompt Set were consulted for structured prompt design and evaluation methodology.